

Decisiones arquitectónicas propuestas

ADR-01: Implementación de una capa Service en el backend

Tipo: Backend

Problema

Durante el análisis del backend se observó que parte de la lógica de negocio se encuentra implementada directamente dentro de los controladores REST.

Decisión

Implementar una **capa Service** que centralice la lógica de negocio del sistema.

Justificación

Esto permite separar claramente las responsabilidades entre controladores, servicios y repositorios, mejorando la mantenibilidad y organización del backend.

Ejemplo para entenderlo

Antes:

Controller ↓ Hace validaciones ↓ Consulta repositorio ↓ Guarda datos

Después:

Controller ↓ Service (lógica de negocio) ↓ Repository

ADR-02: Centralización del acceso a la API en el frontend

Tipo: Frontend

Problema

En aplicaciones Angular múltiples componentes pueden realizar llamadas HTTP directamente, lo que genera duplicación de lógica.

Decisión

Centralizar todas las llamadas a la API en un servicio común (**api.service.ts**).

Justificación

Esto permite mantener una única capa responsable de la comunicación con el backend, facilitando el mantenimiento del frontend.

Ejemplo para entenderlo

Antes:

PeopleComponent → llama API CoursesComponent → llama API PaymentsComponent → llama API

Después:

Components ↓ ApiService ↓ Backend API

ADR-03: Uso de DTO para intercambio de datos

Tipo: Backend

Problema

Las entidades del backend pueden ser utilizadas directamente para enviar información al frontend.

Decisión

Implementar **DTO (Data Transfer Objects)** para transferir información entre el backend y el frontend.

Justificación

Esto desacopla el modelo de persistencia de la representación de datos expuesta en la API.

Ejemplo para entenderlo

Entidad completa:

Usuario

- id
- nombre
- correo
- contraseña
- rol
- fecha_creación

Pero el frontend solo necesita algunos datos.

DTO enviado al frontend:

UserDTO

- nombre
- correo

De esta forma el sistema **no expone todos los datos del usuario**.

ADR-04: Implementación de interceptores HTTP en Angular

Tipo: Frontend

Problema

Las solicitudes HTTP del frontend requieren incluir el token de autenticación en cada petición.

Decisión

Utilizar un **HTTP Interceptor** en Angular para agregar automáticamente el token JWT.

Justificación

Esto centraliza la lógica de autenticación y evita duplicación de código en los componentes.

Ejemplo para entenderlo

Después del login el sistema genera un token.

Sin interceptor:

Componente → agrega token manualmente Otro componente → agrega token manualmente Otro componente → agrega token manualmente

Con interceptor:

Usuario hace login ↓ Se guarda el token ↓ Interceptor agrega automáticamente el token a todas las peticiones

ADR-05: Separación de la lógica de pagos en un servicio especializado

Tipo: Backend

Problema

Las operaciones relacionadas con pagos están distribuidas dentro de diferentes controladores.

Decisión

Crear un **PaymentService** encargado de gestionar todas las operaciones relacionadas con pagos.

Justificación

Esto mejora la organización del sistema y facilita futuras integraciones con servicios externos de pago.

Ejemplo para entenderlo

Antes:

EnrollmentController maneja pagos OfferingController maneja pagos

Después:

Controllers ↓ PaymentService ↓ PaymentRepository

ADR-06: Implementación de paginación en consultas

Tipo: Backend / Frontend

Problema

Las consultas que retornan listas de datos pueden crecer considerablemente cuando el sistema tenga mayor volumen de información.

Decisión

Implementar **paginación en los endpoints REST** y manejarla también en el frontend.

Justificación

Esto mejora el rendimiento del sistema y optimiza la carga de información en la interfaz de usuario.

Ejemplo para entenderlo

Página 1 → registros 1 a 10 Página 2 → registros 11 a 20 Página 3 → registros 21 a 30

Es similar a plataformas como Netflix donde se muestran algunos elementos y luego aparecen más.

ADR-07: Protección de rutas mediante Guards en Angular

Tipo: Frontend

Problema

Las rutas del sistema deben ser accesibles únicamente por usuarios autenticados.

Decisión

Implementar **Auth Guards** en Angular para controlar el acceso a las rutas protegidas.

Justificación

Esto mejora la seguridad del sistema y evita accesos no autorizados a la aplicación.

Ejemplo para entenderlo

Usuario intenta entrar a /dashboard ↓ AuthGuard verifica login ↓ Si no está autenticado → redirige a login

ADR-08: Manejo global de errores en el backend

Tipo: Backend

Problema

El manejo de errores se encuentra distribuido entre diferentes controladores.

Decisión

Implementar un **manejador global de excepciones** utilizando `@ControllerAdvice` en Spring Boot.

Justificación

Esto permite centralizar el manejo de errores y mejorar la consistencia de las respuestas de la API.

Ejemplo para entenderlo

Frontend pide usuario ↓ Backend busca usuario ↓ Usuario no existe ↓ Manejador global de errores ↓ Envía mensaje claro

ADR-09: Documentación de la API mediante Postman

Tipo: Backend / Integración

Problema

La API REST del sistema no cuenta con documentación clara de los endpoints disponibles.

Decisión

Utilizar **Postman** para documentar y organizar los endpoints mediante colecciones.

Justificación

Esto permite registrar y probar los endpoints del sistema y compartir la documentación con otros desarrolladores.

Ejemplo para entenderlo

Colección en Postman:

POST /api/auth/login GET /api/people POST /api/people GET /api/courses GET /api/payments

ADR-10: Uso de contenedores Docker para el despliegue del sistema

Tipo: Infraestructura

Problema

La ejecución del sistema puede variar entre diferentes entornos de desarrollo.

Decisión

Utilizar **Docker** para contenerizar el backend, frontend y base de datos.

Justificación

Esto permite mantener consistencia en los entornos de ejecución y simplifica el despliegue del sistema.

Ejemplo para entenderlo

Docker empaqueta todo el sistema en una "caja" que contiene:

Backend Frontend Base de datos Dependencias

De esta forma cualquier persona puede ejecutar el sistema fácilmente en cualquier computador o servidor.